Getting Started with Adaptive Filters (ADAPTFILT) Objects

This demonstration illustrates how to use the adaptive filter algorithms provided in the Filter Design Toolbox.

We will use a very simple signal enhancement application as an illustration. While we demonstrate with two algorithms only, there are about 30 different adaptive filtering algorithms included with the Filter Design Toolbox.

Contents

- Getting Help
- Desired Signal
- Noise Signal
- Noisy Signal
- Reference Signal
- Constructing Adaptive Filters
- Choosing the Step Size
- Setting the Step Size
- Filtering with Adaptive Filters
- Optimal Solution
- Plot of Results
- Final Coefficients
- Resetting the Filter Before Filtering
- Learning Curves
- Computing the Learning Curves
- Theoretical Learning Curves

Getting Help

To see a list of all algorithms available along with all the methods that are applicable to adaptive filters type "helpwin adaptfilt" or "help adaptfilt".

To get help on a specific algorithm you can type "help adaptfilt/lms" for example.

We now provide a simple example.

Desired Signal

We want to use an adaptive filter to extract a desired signal from a noise corrupted signal by canceling the noise. The desired signal is a sinusoid

```
n = (1:1000)';
s = sin(0.075*pi*n);
```

Noise Signal

We assume that the noise signal v1 is autoregressive.

```
v = 0.8*randn(1000,1);
ar = [1, 1/2];
v1 = filter(1,ar,v);
```

Noisy Signal

The noise corrupted sinusoid is simply s + v1

```
x = s + v1;
```

Reference Signal

We assume we have a moving average signal v2 that is correlated with v1. This will be used as the reference signal.

```
ma = [1, -0.8, 0.4 , -0.2];
v2 = filter(ma,1,v);
```

Constructing Adaptive Filters

We will use two adaptive filters each with 7 taps (weights), an LMS filter, and a normalized LMS (NLMS) filter.

Type "helpwin adaptfilt/lms" and "helpwin adaptfilt/nlms" for further information on constructing LMS and NLMS adaptive filters.

```
L = 7;
hlms = adaptfilt.lms(7);
hnlms = adaptfilt.nlms(7);
```

Choosing the Step Size

All the LMS-like algorithm have a so-called step size which determines the amount of correction that is done as the filter adapts from one iteration to the next. Choosing the appropriate step size is not always easy, a step size that is too small, will hamper convergence speed, while one that is too large may cause the filter to diverge. The Filter Design Toolbox includes algorithms to determine the maximum step size allowed for these algorithms while ensuring convergence.

Type "helpwin adaptfilt/maxstep" for more information.

```
[mumaxlms,mumaxmselms]   = maxstep(hlms,x)
[mumaxnlms,mumaxmsenlms] = maxstep(hnlms); % Always equal to two
```

```
mumaxlms =

      0.21548
```

```
mumaxmselms =

      0.12642
```

Setting the Step Size

The first output of maxstep is the value needed for the coefficients mean to converge, while the second one is the value needed for the mean squared coefficients to converge. However, choosing a large step size also results in large variations from the convergence values, so we choose smaller step sizes

```
hlms.StepSize  = mumaxmselms/30;
hnlms.StepSize = mumaxmsenlms/20;
```

If we already know the step size we want to use, we can also set its value when we first create the filter: hlms = adaptfilt.lms(N,step);

Filtering with Adaptive Filters

Now that we have setup the parameters of the adaptive filters, we can filter the noisy signal. The reference signal, v2 will be the input to the adaptive filters, while x is the desired signal in this setup. The output of the filters, y will try to emulate x as best possible. However, since the input to the adaptive filter, v2, is only correlated with the noise component of x, v1, it can only really emulate this. The error signal, that is the desired x, minus the actual output y will thus constitute an estimate of the part of x that is not correlated with v2 namely s which is the signal we want to extract from x.

Type "helpwin adaptfilt/filter" for a complete description of filtering with adaptive filters.

```
[ylms,elms] = filter(hlms,v2,x);
[ynlms,enlms] = filter(hnlms,v2,x);
```

Optimal Solution

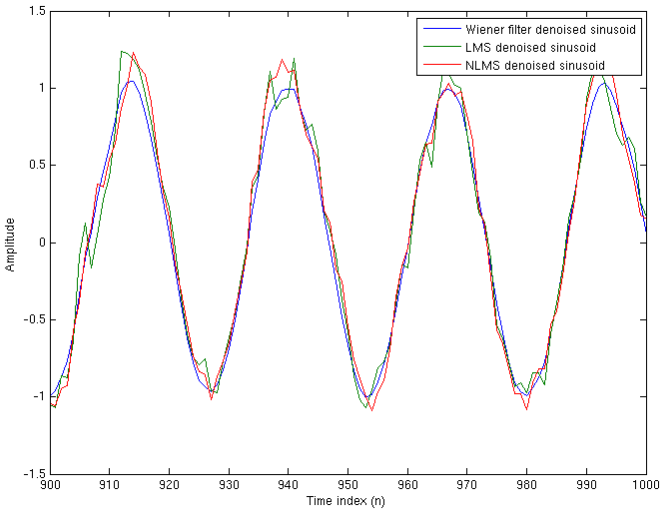
For comparison, we compute the optimal FIR Wiener filter

```
bw = firwiener(L-1,v2,x); % Optimal FIR Wiener filter
yw = filter(bw,1,v2);   % Estimate of x using Wiener filter
ew = x - yw;            % Estimate of actual sinusoid
```

Plot of Results

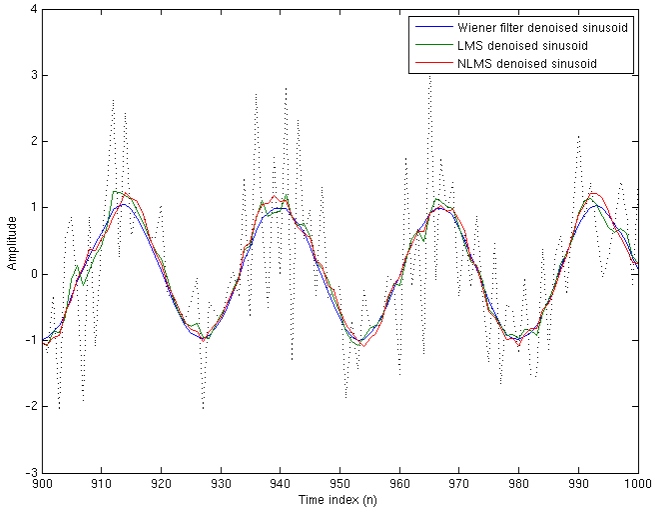
We now show the estimated sinusoid for each case

```
plot(n(900:end),[ew(900:end), elms(900:end),enlms(900:end)]);
legend('Wiener filter denoised sinusoid',...
    'LMS denoised sinusoid', 'NLMS denoised sinusoid');
xlabel('Time index (n)');
ylabel('Amplitude');
```



As a reference, we show the noisy signal dotted

```
hold on
plot(n(900:end),x(900:end),'k:')
xlabel('Time index (n)');
ylabel('Amplitude');
hold off
```



Final Coefficients

We can compare the Wiener filter coefficients with those of the adaptive filters. The adaptive filters will try to converge to the Wiener coefficients

```
[bw.' hlms.Coefficients.' hnlms.Coefficients.']
```

```
ans =

      0.97526      0.86269      0.92527
      0.25828     0.095086      0.17447
      0.04926    -0.042216    -0.051054
   -0.0068056    -0.033636     -0.11745
     0.082471     0.051695     -0.06418
     0.046008   -0.0029945    -0.067949
     0.018264   -0.0086711     -0.03072
```

Resetting the Filter Before Filtering

The adaptive filters have a "PersistentMemory" flag that can be used to reproduce experiments exactly. By default, the flag is false, which means that the states and the coefficients of the filter are reset before filtering. For instance, the following two successive calls produce the same output in both cases.

```
[ylms,elms] = filter(hlms,v2,x);
[ylms2,elms2] = filter(hlms,v2,x);
```

To keep the history of the filter when filtering a new chunk of data, we must set the flag to "off". In this case, the final states and coefficients of a previous run are used as initial conditions for the next set of data.

```
[ylms,elms] = filter(hlms,v2,x);
hlms.PersistentMemory = true;
[ylms2,elms2] = filter(hlms,v2,x); % No longer the same
```

Setting the flag to true can be useful when filtering large amounts of data which can be partitioned into smaller chunks and then fed into the filter using a for-loop.

Learning Curves

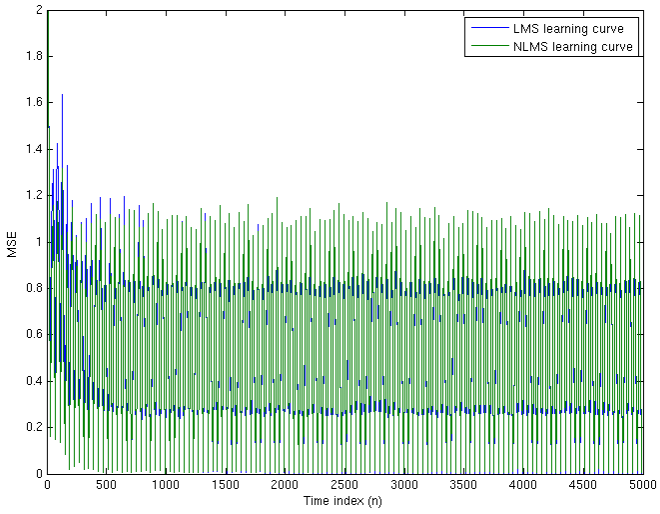
To analyze the convergence of the adaptive filters, we look at the so-called learning curves. These can easily be generated in the Filter Design Toolbox, but we need more experiments to obtain significant results. We will use 25 sample realizations of the noisy sinusoids.

```
n = (1:5000)';
s = sin(0.075*pi*n);
nr = 25;
v = 0.8*randn(5000,nr);
v1 = filter(1,ar,v);
x = repmat(s,1,nr) + v1;
v2 = filter(ma,1,v);
```

Computing the Learning Curves

We now compute the mean-square error. To speed things up, we only compute the error every 10 samples. First we need to reset the adaptive filters to avoid using the coefficients it has already computed and the states it has stored.

```
reset(hlms);
reset(hnlms);
M = 10; % Decimation factor
mselms = msesim(hlms,v2,x,M);
msenlms = msesim(hnlms,v2,x,M);
plot(1:M:n(end),[mselms,msenlms])
legend('LMS learning curve','NLMS learning curve')
xlabel('Time index (n)');
ylabel('MSE');
```

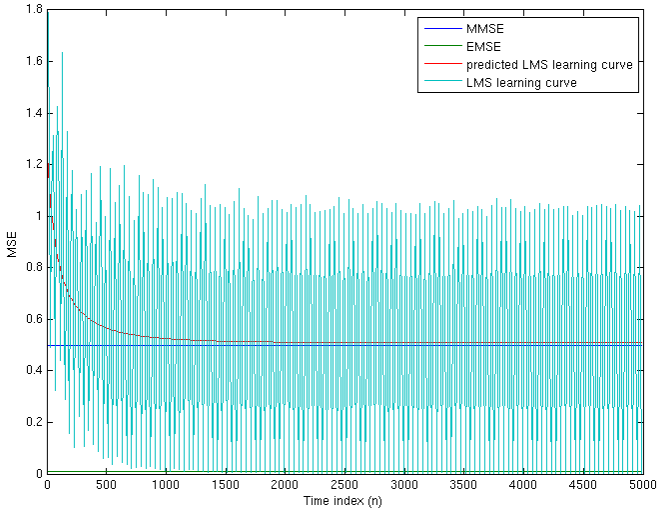


Theoretical Learning Curves

For the LMS and NLMS algorithms, the theoretical learning curves can also be computed, along with the minimum mean-square error (MMSE) the excess mean-square error (EMSE) and the mean value of the coefficients.

This may take a while.

```
reset(hlms);
[mmselms,emselms,meanwlms,pmselms] = msepred(hlms,v2,x,M);
plot(1:M:n(end),[mmselms*ones(500,1),emselms*ones(500,1),...
        pmselms,mselms])
legend('MMSE','EMSE','predicted LMS learning curve',...
    'LMS learning curve')
xlabel('Time index (n)');
ylabel('MSE');
```



Copyright 1999-2005 The MathWorks, Inc.